

```

mysql> -- Queries and Sub-Queries
mysql> /* What is subquery?
/*> a subquery is a query inside another query.
/*> it is written inside parenthesis() and its result is used by the outer
query.
/*> it is also called.
/*> inner Query
/*> Nested Query
/*> inner select.
/*>
/*> purpose of subquery
/*> -----
/*> 1.To break down complex problems into smaller queries.
/*> 2.to use the result of one query as input to another
/*> 3.To avoid multiple steps(write in a single Query insted of temp tables)
/*>
/*>
/*> general syntax
/*> -----
/*> select column-list from table
/*> where column OPERATOR(select column from table where condition);
/*> */
mysql>
mysql> /* types of subqueries
/*> -----
/*> 1. Single-row-subquery
/*> 2. multi-row subquery
/*> 3. nested subquery.
/*> 4. Correlated subquery.
/*> 5. Exists/not Exists subquery */
mysql>
mysql> /* 1. Single-row-subquery:Single row subqueries return exactly one row and
one column. They use comparison operators like =,
/*> >,>=,<,<=,!=
/*>
/*> Characteristics:
/*> -----
/*>
/*> 1.Returns only one value
/*> 2.Can use single-value comparison operators
/*> 3.If subquery returns no rows or multiple rows, it causes an error.
/*>
/*>
/*> Syntax:
/*> select columns
/*> from table
/*> where column operator(select column from table where condition); */
mysql>
mysql> /* 2. multi-row subquery
/*> -----

```

```

/*> 1. multi-row subquery IN
/*> A Multi-Row Subquery returns more than one row. The IN operator is used when
you want to compare a value with multiple values returned by the subquery.
/*>
/*> Characteristics:
/*> -----
/*> 1.Returns multiple rows
/*> 2.Uses the IN operator
/*> 3.Checks whether a value exists in the list returned by the subquery
/*> 4.Equivalent to multiple OR conditions
/*> 5.Most commonly used for matching values
/*>
/*> Syntax:
/*> -----
/*> select column from a table where column IN(select column from table where
condition);
/*>
/*>
/*> 2. Multi-row Subquery (ANY / SOME)
/*> -----
/*> Multi row subqueries return multiple rows.
/*> When using ANY or SOME. the condition is true if it matches at least one
value returned by the subquery.
/*>
/*> Characteristics:
/*> -----
/*> 1.Returns multiple values
/*> 2.ANY and SOME are interchangeable
/*> 3.Condition is true if ANY of the values satisfy the condition.
/*> 4.Often used with IN, >ANY, <ANY, >=ANY, <=ANY .....etc.
/*>
/*> Syntax:
/*> -----
/*> select column from table
/*> where column OPERATOR ANY/SOME(select column from table where condition);
/*>
/*>
/*> 3.Multi-row Subquery ALL
/*> -----
/*> The ALL operator means the condition must be true for ALL values returned by
the subquery.
/*>
/*> Characteristics:
/*> -----
/*> 1.Condition must be true for ALL values in the subquery result
/*> 2.Often used with >ALL, <ALL, >=ALL, <=ALL
/*> 3.>ALL means greater than the maximum value
/*> 4.<ALL means less than the minimum value.
/*>
/*> Syntax:

```

```

/*> -----
/*> select column from table where column Operator all(select column from table
where condition);
/*>
/*>
/*> 3. Nested Subquery
/*> -----
/*> Nested subqueries are subqueries within subqueries. They can be multiple
levels deep and are
/*> executed from the innermost to the outermost.
/*>
/*> Characteristics:
/*> -----
/*> 1. Multiple levels of nesting
/*> 2. Inner queries execute first
/*> 3. Can combine different types of subqueries
/*> 4. Useful for complex conditions
/*>
/*> Syntax:
/*> -----
/*> select column -list from table
/*> where column=(select column from table where column=(select column from
table where condition));
/*>
/*>
/*> 4. Correlated Subquery
/*> -----
/*> Correlated subqueries are subqueries that reference columns from the outer
query.
/*> Unlike regular subqueries that execute once, correlated subqueries execute
once for each row processed by the outer query.
/*>
/*> Characteristics:
/*> -----
/*> 1. References columns from the outer query
/*> 2. Executes multiple times (once per outer row)
/*> 3. Cannot be executed independently
/*> 4. Often used with EXISTS, NOT EXISTS
/*> 5. Generally slower than non-correlated subqueries.
/*>
/*> Syntax:
/*> -----
/*> select column from table1 outer_alis
/*> where condition(select column from table2 inner_alis
/*>                      where inner_alis.column=outer_alis.column);
/*>
/*>
/*> 5. EXISTS / NOT EXISTS Subquery
/*> -----
/*> EXISTS is used to test whether a subquery returns any rows. It returns TRUE

```

if the subquery
/*> returns at least one row, FALSE otherwise. NOT EXISTS returns TRUE if the subquery returns no rows.

```
/*>  
/*> Characteristics:  
/*> -----  
/*> 1.Returns TRUE/FALSE (boolean result)  
/*> 2.More efficient than IN for large datasets  
/*> 3.Often used with correlated subqueries  
/*> 4.Stops execution as soon as first match is found  
/*> 5.SELECT clause in EXISTS doesn't matter (often use SELECT 1)  
/*>  
/*>  
/*> Syntax:  
/*> -----  
/*> select columns from table1  
/*> where exists(select 1 from table2 where condition).  
/*>  
/*>  
/*>  
/*> Syntax:  
/*> -----  
/*> select columns from table1  
/*> where not exists(select 1 from table2 where condition).  
/*> */
```

```
mysql>  
mysql>  
mysql> drop table products;  
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> drop table orders;  
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that  
corresponds to your MySQL server version for the right syntax to use near 'drope  
table orders' at line 1  
mysql> drop table orders;  
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> -- Step 1: Create the tables  
mysql> CREATE TABLE Customers (  
-> customer_id INT PRIMARY KEY,  
-> customer_name VARCHAR(50),  
-> city VARCHAR(50),  
-> customer_type VARCHAR(20)  
-> );  
Query OK, 0 rows affected (0.03 sec)
```

```
mysql>  
mysql> CREATE TABLE Products (  
-> product_id INT PRIMARY KEY,  
-> product_name VARCHAR(50),
```

```
-> category      VARCHAR(30),
-> price          DECIMAL(10,2)
-> );
```

Query OK, 0 rows affected (0.03 sec)

mysql>

mysql> CREATE TABLE Orders (

```
-> order_id       INT PRIMARY KEY,
-> customer_id    INT REFERENCES Customers(customer_id),
-> product_id     INT REFERENCES Products(product_id),
-> quantity       INT,
-> order_date     DATE,
-> amount         DECIMAL(10,2)
-> );
```

Query OK, 0 rows affected (0.02 sec)

mysql>

mysql> -- Step 2: Insert Customers records

mysql> INSERT INTO Customers (customer_id, customer_name, city, customer_type)
VALUES (1, 'Rahul', 'Delhi', 'Premium');

Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Customers (customer_id, customer_name, city, customer_type)
VALUES (2, 'Meena', 'Mumbai', 'Regular');

Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Customers (customer_id, customer_name, city, customer_type)
VALUES (3, 'Arjun', 'Bangalore', 'Premium');

Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO Customers (customer_id, customer_name, city, customer_type)
VALUES (4, 'Kavya', 'Delhi', 'Regular');

Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO Customers (customer_id, customer_name, city, customer_type)
VALUES (5, 'Suresh', 'Chennai', 'Premium');

Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO Customers (customer_id, customer_name, city, customer_type)
VALUES (6, 'Anjali', 'Mumbai', 'Regular');

Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO Customers (customer_id, customer_name, city, customer_type)
VALUES (7, 'Vikram', 'Delhi', 'Regular');

Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO Customers (customer_id, customer_name, city, customer_type)
VALUES (8, 'Pooja', 'Bangalore', 'Premium');

Query OK, 1 row affected (0.00 sec)

```
mysql> INSERT INTO Customers (customer_id, customer_name, city, customer_type)
VALUES (9, 'Devi', 'Chennai', 'Regular');
Query OK, 1 row affected (0.00 sec)
```

```
mysql>
```

```
mysql> -- Step 3: Insert Products records
```

```
mysql> INSERT INTO Products (product_id, product_name, category, price) VALUES (1,
'Laptop', 'Electronics', 55000);
Query OK, 1 row affected (0.01 sec)
```

```
mysql> INSERT INTO Products (product_id, product_name, category, price) VALUES (2,
'Mobile', 'Electronics', 25000);
Query OK, 1 row affected (0.01 sec)
```

```
mysql> INSERT INTO Products (product_id, product_name, category, price) VALUES (3,
'Chair', 'Furniture', 7000);
Query OK, 1 row affected (0.01 sec)
```

```
mysql> INSERT INTO Products (product_id, product_name, category, price) VALUES (4,
'Table', 'Furniture', 9000);
Query OK, 1 row affected (0.01 sec)
```

```
mysql> INSERT INTO Products (product_id, product_name, category, price) VALUES (5,
'Headphones', 'Electronics', 3000);
Query OK, 1 row affected (0.00 sec)
```

```
mysql> INSERT INTO Products (product_id, product_name, category, price) VALUES (6,
'Bookshelf', 'Furniture', 6000);
Query OK, 1 row affected (0.01 sec)
```

```
mysql>
```

```
mysql> -- Step 4: Insert Orders records
```

```
mysql> INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date,
amount) VALUES (1, 1, 1, 1, '2023-01-05', 55000);
Query OK, 1 row affected (0.01 sec)
```

```
mysql> INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date,
amount) VALUES (2, 2, 2, 1, '2023-01-10', 25000);
Query OK, 1 row affected (0.01 sec)
```

```
mysql> INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date,
amount) VALUES (3, 3, 3, 2, '2023-02-01', 14000);
Query OK, 1 row affected (0.00 sec)
```

```
mysql> INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date,
amount) VALUES (4, 1, 5, 2, '2023-02-15', 6000);
Query OK, 1 row affected (0.00 sec)
```

```
mysql> INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date,
amount) VALUES (5, 4, 4, 1, '2023-03-03', 9000);
```

Query OK, 1 row affected (0.01 sec)

```
mysql> INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date, amount) VALUES (6, 5, 2, 2, '2023-03-20', 50000);
```

Query OK, 1 row affected (0.00 sec)

```
mysql> INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date, amount) VALUES (7, 6, 4, 3, '2023-04-02', 18000);
```

Query OK, 1 row affected (0.00 sec)

```
mysql> INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date, amount) VALUES (8, 3, 5, 1, '2023-04-15', 3000);
```

Query OK, 1 row affected (0.01 sec)

```
mysql> INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date, amount) VALUES (9, 7, 3, 1, '2023-05-01', 7000);
```

Query OK, 1 row affected (0.01 sec)

```
mysql> INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date, amount) VALUES (10, 8, 1, 1, '2023-05-20', 54000);
```

Query OK, 1 row affected (0.01 sec)

```
mysql> INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date, amount) VALUES (11, 2, 3, 2, '2023-06-02', 12000);
```

Query OK, 1 row affected (0.00 sec)

```
mysql> INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date, amount) VALUES (12, 5, 4, 1, '2023-06-15', 8500);
```

Query OK, 1 row affected (0.01 sec)

```
mysql> select * from Customers;
```

customer_id	customer_name	city	customer_type
1	Rahul	Delhi	Premium
2	Meena	Mumbai	Regular
3	Arjun	Bangalore	Premium
4	Kavya	Delhi	Regular
5	Suresh	Chennai	Premium
6	Anjali	Mumbai	Regular
7	Vikram	Delhi	Regular
8	Pooja	Bangalore	Premium
9	Devi	Chennai	Regular

9 rows in set (0.00 sec)

```
mysql> select * from products;
```

product_id	product_name	category	price
------------	--------------	----------	-------

1	Laptop	Electronics	55000.00
2	Mobile	Electronics	25000.00
3	Chair	Furniture	7000.00
4	Table	Furniture	9000.00
5	Headphones	Electronics	3000.00
6	Bookshelf	Furniture	6000.00

6 rows in set (0.00 sec)

mysql> select * from orders;

order_id	customer_id	product_id	quantity	order_date	amount
1	1	1	1	2023-01-05	55000.00
2	2	2	1	2023-01-10	25000.00
3	3	3	2	2023-02-01	14000.00
4	1	5	2	2023-02-15	6000.00
5	4	4	1	2023-03-03	9000.00
6	5	2	2	2023-03-20	50000.00
7	6	4	3	2023-04-02	18000.00
8	3	5	1	2023-04-15	3000.00
9	7	3	1	2023-05-01	7000.00
10	8	1	1	2023-05-20	54000.00
11	2	3	2	2023-06-02	12000.00
12	5	4	1	2023-06-15	8500.00

12 rows in set (0.00 sec)

mysql> -- find the name and price of the most expensive product.

mysql> select product_name,price from products where price = (select max(price) from products);

product_name	price
Laptop	55000.00

1 row in set (0.00 sec)

mysql> -- find the order(order_id,customer_id,amount) with the smallest order amount.

mysql> select order_id,customer_id,amount from orders where amount =(select min(amount) from orders);

order_id	customer_id	amount
8	3	3000.00

1 row in set (0.00 sec)

mysql> -- find the customer name who placed order number 5

```
mysql> select customer_name from customers where customer_id=(select customer_id
from orders where order_id=5);
```

```
+-----+
| customer_name |
+-----+
| Kavya         |
+-----+
```

```
1 row in set (0.00 sec)
```

```
mysql> -- Find the category of the product that was purchased in order number 1.
```

```
mysql> select category from products where product_id =(select product_id from
orders where order_id=1);
```

```
+-----+
| category   |
+-----+
| Electronics|
+-----+
```

```
1 row in set (0.00 sec)
```

```
mysql> -- Find the name of the product involved in the order with the highest
amount.
```

```
mysql> select product_name from products where product_id=(select product_id from
orders where amount =(select max(amount) from orders));
```

```
+-----+
| product_name |
+-----+
| Laptop       |
+-----+
```

```
1 row in set (0.00 sec)
```

```
mysql> -- Find the names of customers who live in 'Delhi' or 'Mumbai'.
```

```
mysql> select customer_name from customers where city in('Delhi','Mumbai');
```

```
+-----+
| customer_name |
+-----+
| Rahul         |
| Meena        |
| Kavya        |
| Anjali       |
| Vikram       |
+-----+
```

```
5 rows in set (0.00 sec)
```

```
mysql> -- Find the names of products that have been ordered at least once.
```

```
mysql> select product_name from products where product_id in (select product_id
from orders);
```

```
+-----+
| product_name |
+-----+
```

Laptop
Mobile
Chair
Table
Headphones

+-----+

5 rows in set (0.00 sec)

mysql> -- Find the names of customers who have placed at least one order.

mysql> select customer_name from customers where customer_id in(select customer_id from orders);

+-----+

customer_name

+-----+

Rahul
Meena
Arjun
Kavya
Suresh
Anjali
Vikram
Pooja

+-----+

8 rows in set (0.00 sec)

mysql> -- find products that have never been ordered.

mysql> select product_name from products where product_id not in (select product_id from orders);

+-----+

product_name

+-----+

Bookshelf

+-----+

1 row in set (0.00 sec)

mysql> -- Find customers whose city matches a city where at least one 'Premium' customer lives.

mysql> select customer_name from customers where city in(select city from customers where customer_type='Premium');

+-----+

customer_name

+-----+

Rahul
Arjun
Kavya
Suresh
Vikram
Pooja
Devi

+-----+

7 rows in set (0.00 sec)

mysql> -- Find products priced greater than ANY product in the 'Furniture' category.

mysql> select product_name ,price from products where price > any(select price from products where category= 'Furniture');

product_name	price
Laptop	55000.00
Mobile	25000.00
Chair	7000.00
Table	9000.00

4 rows in set (0.00 sec)

mysql> -- find products priced less than some products in electronic category.

mysql> select product_name ,price from products where price < any(select price from products where category= 'electronics');

product_name	price
Mobile	25000.00
Chair	7000.00
Table	9000.00
Headphones	3000.00
Bookshelf	6000.00

5 rows in set (0.00 sec)

mysql> -- find orders with an amount greather than any order placed by 'arjun'

mysql> select order_id,amount from orders where amount >any(select amount from orders where customer_id=(select customer_id from customers where customer_name='Arjun'));

order_id	amount
1	55000.00
2	25000.00
3	14000.00
4	6000.00
5	9000.00
6	50000.00
7	18000.00
9	7000.00
10	54000.00
11	12000.00
12	8500.00

11 rows in set (0.00 sec)

```
mysql> -- Find orders with an amount less than ANY order placed by a customer from 'Mumbai'.
```

```
mysql> select order_id,amount from orders where amount <any(select amount from orders where customer_id in (select customer_id from customers where city='Mumbai'));
```

order_id	amount
3	14000.00
4	6000.00
5	9000.00
7	18000.00
8	3000.00
9	7000.00
11	12000.00
12	8500.00

```
8 rows in set (0.00 sec)
```

```
mysql> notee
```